Security Documentation

EtelPay Platform

Prepared for INSA Cyber-Security Audit Application (submitted via the Hibir platform)

Etel-FTBG · 2026-09-24

INSA Pre-Submission Security Findings — EtelPay

First-time INSA pre-submission audit of EtelPay (Etel Group's digital wallet), living at `Etel.Pay/wallet/` in this repo, deployed at `ethelmarket.com/Etel.Pay/wallet/`. No prior audit document existed for this system — this file is written from scratch. Scope covers every PHP file under `Etel.Pay/wallet/api/`, the frontend HTML/JS under `Etel.Pay/wallet/`, and `Etel.Pay/wallet/backups/`. No deployment was performed — everything below is a code-level fix in the git working tree. Deployment to `ethelmarket.com/Etel.Pay/wallet/` happens separately, through EtelPay's own Updater tool (`updater.html`), uploaded manually by Dagi.

Every finding below was produced by reading the actual code (file/function/action names cited throughout), not inferred. Every `.php` file edited was verified with `php -l` (no syntax errors) and checked for **new** non-ASCII bytes introduced by the edit (`python3 -c "print([ch for ch in open(f).read() if ord(ch)>127])"`, diffed against what was already in the file) — EtelPay is one of the two systems (with EtelID) under this host's hard ASCII-purity constraint, since this host's WAF has previously corrupted non-ASCII bytes in PHP source. Pre-existing Amharic text and box-drawing comment characters throughout this codebase are left as-is; no new ones were introduced by any edit in this pass.

EtelPay is architecturally distinct from Delivery & Ride and EtelGebeya: it is a single flat router (`v2_verify.php`, deliberately not named `api.php` — see the shared ops skill notes on this host's WAF silently deleting files named `api.php` or over ~30KB) that requires in three route-handler files (`ep_r1.php`, `ep_r2.php`, `ep_r3.php`, together defining `ep_route_part1/2/3()`), an EtelID bridge (`etelid_kyc.php`), and several standalone service-to-service scripts that bypass the router entirely (`etelid_member_detail.php`, `etelid_members_list.php`, `service_check_wallet.php`, `service_nfc_charge.php`). There is no MVC controller layer, no ORM, no `boot.php`-style per-file action whitelist — every `case '<action>':` inside the three `ep_route_part*()` switch statements is the entire access-control surface for that action.

---

1. Authorization enforcement

Checked: every `case` in `ep_r1.php` (832 lines), `ep_r2.php` (460 lines), `ep_r3.php` (453 lines) — roughly 80 distinct action names — for whether the action requires a valid session (`ep_auth()` / `ep_auth_id()`), admin privilege (`_ep_require_admin()`), or a service-to-service shared secret (`hash_equals(EP_SECRET_FINAL, ...)` / `hash_equals(ETELID_SERVICE_KEY, ...)`), and whether any per-row ownership check exists wherever a request supplies an ID belonging to another user's data (IDOR class). Also checked the four standalone service scripts and the KYC bridge for the same.

Fixed — Item 1: four money-moving endpoints checked the request key against a

possibly-still-placeholder `EP_SECRET` instead of the safe, auto-generated `EP_SECRET_FINAL` (already fixed in this session, ahead of this pass)

Finding (Critical/High): `ep_r1.php`'s four service-to-server, money-moving actions — `ep_escrow_release`, `ep_escrow_reverse`, `ep_verify_tx` / `etel_pay_verify_tx`, and `ep_partner_credit`, plus (added in the same fix) `ep_platform_transfer` — authenticate the caller by comparing an `X-Etel-Key` header (or `?key=`) against a shared secret with `hash_equals()`. They were comparing against the raw `EP_SECRET` constant, which `config.php` ships as the literal placeholder string `'CHANGE_ME'`. `v2_verify.php` (lines ~137–150) already

computes a safe alternative specifically for this scenario: `EP_SECRET_FINAL` — if `EP_SECRET` is still the shipped placeholder (checked via `ep_is_placeholder()`, a case-insensitive check against `'change_me'` / `'changeme'` / `'change-me'`), it auto-derives a per-server value (`hash('sha256', php_uname('n') . EP_DB_NAME . EP_DB_USER . 'etelpay2026')`) instead of using the known, guessable literal. `EP_SECRET_FINAL` is defined before `ep_r1.php` is required (`v2_verify.php` line 334 loads `etelid_kyc.php`, then `ep_r1.php`, after the `EP_SECRET_FINAL` block at lines 137–153), so it was already available at every one of these call sites. Using the raw `EP_SECRET` meant that on any deployment where the operator never edited `config.php`'s shipped default, all five real-money actions (release/reverse an escrow hold, verify a transaction server-side, credit a partner payout, or move money directly between two wallets) would accept the literal string `CHANGE_ME` as a valid service key — full unauthenticated money-movement on a live fintech system.

Fix (already applied, verified here): all five `hash_equals(EP_SECRET, (string)$key)` call sites in `ep_r1.php` now read `hash_equals(EP_SECRET_FINAL, (string)$key)`, with the surrounding comments updated to explain why (`ep_r1.php` lines ~382, ~391, ~432–436, ~442, ~464, ~482, ~574, ~591). Re-verified in this pass: `php -l ep_r1.php` reports no syntax errors, and the ASCII-byte scan shows no non-ASCII characters were introduced by this or any other edit in this pass (the file's existing Amharic/box-drawing comment characters are pre-existing and unaffected).

Fixed — Item 2: EtelPay's own file-deploy tool (`updater-api.php`) treated its

own shipped placeholder password as a valid, configured password (Critical)

Finding: `updater-api.php` is the backend for `updater.html` — an arbitrary-file-write tool (`upload_file`, `update_file`, `upload_zip`, `upload_multiple`, `run_sql`) gated only by a single shared password, `EP_UPDATER_PASS`. `config.php` ships this as the literal placeholder `'CHANGE_ME'`. The gate function, `upd_password_is_configured($pass)`, existed specifically to reject a still-placeholder password from ever being treated as "configured" (its own comment says so: *"anyone who knows the public default password could self-sign a valid token ... and hit any authenticated action directly"*) — but its actual check was `$pass !== 'changeme' && $pass !== 'change_this_updater_password'`, a **case-sensitive** comparison against the lowercase `'changeme'`. Since `config.php`'s real shipped literal is `'CHANGE_ME'` (uppercase), `'CHANGE_ME' !== 'changeme'` is true, so the function returned `true` — the exact bypass its own comment warns about. On any deployment where `config.php` was never edited, logging in to the Updater with the literal password `CHANGE_ME` would succeed, granting arbitrary PHP file write (i.e., full remote code execution) on this production fintech host, plus unrestricted raw SQL execution via the `run_sql` action.

Fix applied in `Etel.Pay/wallet/api/updater-api.php`'s `upd_password_is_configured()`: now lower-cases and trims before comparing, and also rejects `'change-me'` for parity with `v2_verify.php`'s `ep_is_placeholder()`:

```
function upd_password_is_configured($pass) {
    $p = strtolower(trim((string)$pass));
    return strlen($p) >= 4 && $p !== 'changeme' && $p !== 'change_this_updater_password' &&
}
```

This single function gates both the `login` action and (per its own existing comment) `upd_verify_token()` for every other authenticated action, so the fix closes the gap everywhere it mattered in one place, matching the file's

existing structure. Verified with `php -l` (no syntax errors) and the ASCII-byte scan (no new non-ASCII bytes; the file's existing Amharic/box-drawing comment characters are unaffected).

Fixed — Item 3: `ep_admin_dispute_resolve`'s refund path could double-process on a

retried/double-clicked request (Medium — double-crediting)

Finding: every other admin money-moving action in this codebase that flips a status and then moves a balance (`ep_admin_reverse_tx`, `ep_admin_topup_approve`) does the status flip **first**, as a single conditional `UPDATE ... WHERE status = 'X'`, and only proceeds to touch any balance if `rowCount() > 0` — both call sites have an explicit code comment explaining this is deliberate, to prevent two concurrent/retried requests from both passing an earlier plain `SELECT` and double-processing the same completed transaction. `ep_admin_dispute_resolve`'s `refund` branch (`ep_r3.php`) was missing this guard entirely: it read the dispute row with a plain `SELECT`, then unconditionally credited the original sender and debited the receiver, and only updated the dispute's `status` to `'resolved'` at the very end of the function. A double-click on the "Refund" button in the admin UI, or any retried request (including this app's own known offline-retry-queue behavior documented in the sibling Delivery & Ride audit), would credit the sender and debit the receiver a second time for the same dispute.

Fix applied in `Etel.Pay/wallet/api/ep_r3.php`'s `ep_admin_dispute_resolve`: the `refund` branch now performs the same atomic status-flip-then-credit pattern as `ep_admin_reverse_tx` / `ep_admin_topup_approve` — `UPDATE ... ep_disputes SET status = "resolved" WHERE id = ? AND status IN ("open","investigating")`, and only proceeds to move any balance if that UPDATE actually affected a row; a second call on an already-resolved dispute now fails cleanly with `'Dispute already resolved/rejected'` instead of moving money again. Verified with `php -l` and the ASCII-byte scan (no new non-ASCII bytes).

Verified clean

- **No client-supplied "role" or "is_admin" field anywhere.** `is_admin` is read everywhere (`ep_get_balance`, `_ep_require_admin()`) but **never written** by any action in `ep_r1.php` / `ep_r2.php` / `ep_r3.php` — grepped for every occurrence. The only way an account becomes an admin is a manual database change (by Dagi, directly or via the Updater's `run_sql` action) or the `admin_phones` setting fallback in `_ep_require_admin()` (`ep_r3.php`), which is itself only ever written by `ep_admin_save_settings` — an action already gated behind `_ep_require_admin()`. This is the exact class of bug the task brief specifically asked to check for (mirroring Delivery & Ride's `AuthController.php` `register` finding) — EtelPay has no such gap because **it has no client-facing self-registration action at all**: `register.html` calls an action named `etel_pay_register_wallet` (line 116) that does not exist anywhere in `ep_r1.php` / `ep_r2.php` / `ep_r3.php`'s switch statements — it would 404 with "Unknown action" if actually called. The only ways a wallet account is created are (a) `ep_login_etelid` (`ep_r3.php`), which auto-provisions a wallet from an already-verified EtelID identity with no way for the caller to influence `is_admin`, `kyc_status` (hardcoded `'verified'`), or any other privileged field, and (b) direct admin action (`ep_admin_credit` etc. operate on existing users only — there is no admin-create-user action either). **`register.html`'s dead registration form is a real functional gap (self-registration is effectively non-working), flagged below under Section 7 for Dagi, but it is not a security hole — if anything, its absence is why this system has no client-controlled-privilege registration bug to find.**
- **`_ep_require_admin()`** (`ep_r3.php`) is the single admin gate used by all 20 `ep_admin_*` / `ep_analytics` actions across `ep_r2.php` / `ep_r3.php` — checked the session token's `is_admin` column first, falling back

to a comma-separated `admin_phones` setting, never trusting any request-body field for the privilege decision.

- **IDOR / ownership scoping** — every user-facing action was checked for whether it trusts a request-supplied ID for another user's data:
- `ep_get_balance`, `ep_get_history`, `ep_get_profile`, `ep_update_profile`, `ep_change_pin`, `ep_upload_avatar`, `ep_my_analytics`, `ep_paylink_list`, `ep_apikey_list`, `ep_kyc_submit` — all resolve the acting user's own ID from `ep_auth_id()` / `ep_auth()` (the session token), never from a request parameter.
- `ep_send` / `ep_escrow_send` / `ep_qr_process` / `ep_paylink_pay` — sender/payer is always `ep_auth_id()`; receiver/QR-owner/paylink-owner is resolved server-side by phone/token lookup, never by a client-supplied user ID.
- `ep_apikey_revoke` scopes its `UPDATE` to `WHERE id=? AND user_id=?` — cannot revoke another user's key even by guessing a `key_id`.
- `ep_topup_confirm` scopes its `SELECT` to `WHERE ref=? AND user_id=?` — cannot confirm another user's top-up request by guessing a `ref`.
- **Concurrent-balance-mutation guards.** Every real debit in this codebase (`ep_send`, `ep_escrow_send`, `ep_qr_process`, `ep_platform_transfer`, `service_nfc_charge.php`) uses the same pattern: `UPDATE ... SET balance_etb = balance_etb - ? WHERE id = ? AND balance_etb >= ?` followed by an explicit `SELECT ROW_COUNT()` check, throwing (and rolling back the whole transaction) if zero rows were affected — the code comments in `ep_send` / `ep_qr_process` correctly explain this guards against two concurrent debits both passing an earlier balance pre-check. This pattern is applied consistently everywhere a balance is debited, including the standalone `service_nfc_charge.php`.
- **Idempotency on all service-to-service money endpoints.** `ep_partner_credit` and `ep_platform_transfer` both require a caller-supplied `partner_ref`, enforced unique per partner via a real DB `UNIQUE KEY` constraint (`uk_partner_ref`), and both **reserve the idempotency slot with an `INSERT` before crediting/transferring any balance** — the code comments correctly note this is what makes a concurrent/retried call impossible to double-process, not just the earlier `SELECT`-based check. Both also enforce a per-call cap and a rolling 24-hour cap per partner.
- **`service_check_wallet.php` and `service_nfc_charge.php`** — per the task brief, independently re-verified: both correctly reject the `ETELID_SERVICE_KEY` placeholder (`strcasecmp(trim(...), 'CHANGE_ME') === 0` — case-insensitive, correctly catching the real shipped default, unlike the updater bug above) before accepting any call, and both use `hash_equals()` for the actual key comparison. `service_check_wallet.php` deliberately does **not** select `balance_etb` / `balance_usd` in its response — its own comment explains this is intentional: a traffic officer's roadside NFC card check has no legitimate need to see a driver's wallet balance, and exposing it there would be a real privacy overreach distinct from what the rest of the card's fields show. `service_nfc_charge.php`'s header comment documents its trust model clearly: it never re-derives payer consent itself — it trusts that EtelID has already independently verified both the receiver's and the payer's own EtelID passwords before ever calling in, the same trust-delegation shape already established (and previously verified clean) for Delivery & Ride's `NfcController.php` in the sibling audit.
- **`etelid_member_detail.php`** already correctly rejects the `CHANGE_ME` placeholder (case-insensitive `strcasecmp`) exactly as the task brief expected, confirmed by reading the file directly (line 52).
- **`etelid_kyc.php`** (the outbound EtelID bridge) guards against direct access (`if (!defined('EP_DB_HOST')) exit;`) and always sends `CURLOPT_SSL_VERIFYPEER => true` on its outbound calls to EtelID — not disabled for convenience.

Needs Dagi's judgment call

- **`ep_admin_self_topup`** (`ep_r2.php`) lets any account holding `is_admin` mint new balance into their own wallet from nowhere, with no counterparty debit — by design (mirrors `ep_admin_credit`, which does the same for any target user). This is a legitimate admin power in a system that also has real Telebirr/CBE/cash top-up flows feeding the same balance from outside the ledger, not a bug — but it does mean any compromised or careless admin credential is a direct path to unlimited self-funding. Not changed here since it's clearly an intentional admin capability, not an oversight; flagging in case Dagi wants a secondary approval step or an audit-log alert on this specific action given its blast radius.
- **`run_sql` in `updater-api.php`** grants an authenticated Updater session arbitrary raw SQL execution (with a soft block on bare `DROP TABLE` / `TRUNCATE` unless `force:true` is passed) against the production database. This is consistent with what this class of single-operator deploy tool is for across the whole Etel Group codebase (matches Delivery & Ride's equivalent tooling), and is now properly gated by the placeholder-fix in Item 2 above — but it remains a very high-blast-radius capability behind a single shared password with no second factor. Not changed here (no established pattern in this codebase for adding MFA to this tool), flagged for awareness.

---

2. SQL injection review

Checked: every `$pdo->prepare()` / `$pdo->query()` / `$pdo->exec()` call across all eleven PHP files in `Etel.Pay/wallet/api/` (`v2_verify.php`, `ep_r1.php`, `ep_r2.php`, `ep_r3.php`, `etelid_kyc.php`, `etelid_member_detail.php`, `etelid_members_list.php`, `service_check_wallet.php`, `service_nfc_charge.php`, `updater-api.php`, `ep_login_rate_limited` / `ep_ensure_login_attempts_table` in `v2_verify.php`) — traced every place a variable is concatenated into a SQL string back to its source.

Findings

None. No SQL injection was found in any request-input-reachable query.

Verified clean

- **Every value from `$_body` / `$_GET` / `$_POST` reaches SQL only via a bound `?` placeholder**, using real prepared statements (`PDO::ATTR_EMULATE_PREPARES => false` is set in every one of this codebase's four independent `new PDO(...)` construction sites — `v2_verify.php`'s `ep_db()`, `service_check_wallet.php`, `service_nfc_charge.php`, and each standalone `etelid_member*.php` / `etelid_members_list.php` file).
- **The only place a variable is concatenated directly into a query string is table names** (`EP_TBL_USERS`, `EP_TBL_TX`, etc.) — every one of these is a `define()`d constant built from `EP_TBL_PREFIX`, itself sourced only from `config.php` / `.env` /a hardcoded default, never from request input.
- **`ep_qr_generate`'s dynamic INSERT** (`ep_r1.php`) builds its column list from a `DESCRIBE` probe of the table's actual schema (`$qcols`), with the field *names* always drawn from a hardcoded list of known candidate columns (`amount`, `currency`, `label`, `expires_at`, `status`, `used`, `created_at`) checked against that probe — never from request input. Only the *values* bound into the resulting placeholders come from the request.

- **`ep_r3.php`'s `_ep_set_setting` / `ep_admin_save_settings`** iterate a hardcoded whitelist array (`platform_name`, `support_phone`, `support_email`, `daily_limit`, `platform_fee`) for which settings keys can be written — the request body only ever supplies the value, never the key name used in the query.
- **`updater-api.php`'s `run_sql` action** is a deliberate, admin-tool exception to "never build SQL from request input" — it exists specifically to let an authenticated operator run arbitrary SQL (schema migrations, one-off fixes) through the Updater UI. This is a documented, intentional capability of the tool itself (see Section 1's "Needs Dagi's judgment call" above), not an injection vulnerability in the ordinary sense — the "injection" is the tool's actual purpose, gated by the Updater password.
- **`etelid_members_list.php`'s `LIMIT`** clause interpolates `(int) $limit` directly (`'... LIMIT ' . (int) $limit`) rather than a bound placeholder — safe because the PHP `(int)` cast happens before concatenation, so the interpolated value can only ever be an integer literal, never attacker-controlled SQL syntax.

Needs Dagi's judgment call

None — no SQL injection surface needing a decision was found in any of the eleven files.

---

3. XSS / output escaping

Checked: every `innerHTML =` / `.html(` / `document.write` / `insertAdjacentHTML` sink across every `.html` page at the wallet root (`admin.html`, `wallet.html`, `history.html`, `send.html`, `profile.html`, `qr.html`, `kyc.html`, `pay.html`, `pay-links.html`, `topup.html`, `login.html`, `register.html`, `updater.html`, `diag.html`, `analytics.html`) and every file under `Etel.Pay/wallet/js/` (`core.js`, `api.js`, `config.js`, `desktop-shell.js`, `desktop.js`, `devpanel.js`, `logo-anim.js`) — roughly 90 sink occurrences found; every one that renders a field that can originate from another user's account (name, note, transaction label, backup filename) was individually traced.

Findings

None requiring a fix. This codebase already applies a single, consistently-used escaping helper (`EP_CORE.esc()` in `js/core.js`, a `textContent`-round-trip escaper identical in shape to Delivery & Ride's `escapeHtml()`) everywhere a field that could contain another user's free text is placed into `innerHTML`.

Verified clean

- **`admin.html`** — the panel most analogous to Delivery & Ride's admin XSS findings (Section 3 of `INSA_FINDINGS.md`), since it renders `full_name` / `phone` / `email` for every user in the system to a privileged viewer. Every render site checked (`renderUserTable`-equivalent at line 434, service-giver-equivalent list at line 477, dispute list at line 510, user-detail card at line 614) wraps the field in `EP_CORE.esc(...)` before interpolating into a template literal. Backup filenames (line 944) are also escaped for display, and the corresponding `onclick="restoreBackup( '${b.name...}')"` string builder (lines 947–948) additionally strips `'` via `.replace(/'/g,"\\'")` — the same defense-in-depth JS-string-hardening pattern Delivery & Ride's audit applied to its own `onclick` builders.
- **`history.html`** — `EP_CORE.txLabel(tx)` (which resolves to `tx.sender_name` or `tx.receiver_name` — another user's `full_name`, fully free text via `ep_update_profile`) is wrapped in `EP_CORE.esc(...)` at the

actual render site (`js` inline in `history.html` line 219), as is `tx.note` (line 220, also user-supplied free text via `ep_send`'s `note` parameter). The core helper function `txLabel()` itself (`js/core.js` lines 176–179) returns the raw, unescaped string — escaping is correctly applied once, at the single point it's rendered into HTML, not relied upon inside the helper.

- **`profile.html`, `wallet.html`, `kyc.html`, `pay.html`** — every place `full_name` / recipient name / EteIID member name is displayed uses `.textContent =`, which is safe by construction regardless of content (confirmed by direct grep — no `innerHTML` assignment of these fields anywhere in these four files).
- **`wallet.html`'s avatar `innerHTML`** (`<img src="${j.data.avatar_url}">`, lines 340 and 528) is not attacker-controllable free text: `avatar_url` is only ever written by `ep_upload_avatar` (`ep_r2.php`), which generates the filename entirely server-side (`'av_' . $uid . '_' . time() . '.' . $ext`, `$ext` restricted to `png`/`jpg` by a PNG magic-byte check) — the client never supplies any part of the stored URL.
- **`js/devpanel.js`** (an in-app developer console overlay, not part of the normal user flow) builds `innerHTML` from its own captured `console.log`/network-request history — self-reflective debugging data the developer generated themselves in their own browser session, not another user's data; same "self-XSS-only, developer tool" shape already judged low-risk for Delivery & Ride's `audit_agent.php` in the sibling audit.
- **`diag.html`**'s `log()` helper (line 98) builds `innerHTML` from `JSON.stringify()` of raw API responses without escaping `<`/`>` inside that JSON — but this is a manual, developer-run connectivity/diagnostics console (not linked from any normal user-facing page, and every request on it is one the developer running it just issued themselves), the same low-risk shape as the two tools above. Documented here rather than silently passed over, but not fixed — see "Needs Dagi's judgment call" below.

Needs Dagi's judgment call

- **`diag.html`'s unescaped `JSON.stringify()`-into-`innerHTML` pattern** (above) is low-risk today (self-triggered, developer-only page, not linked from the app's normal navigation), but if this page is ever left reachable to less-trusted users, or ever starts rendering a response that echoes another party's input (e.g., an error message that includes a phone number or name someone else supplied), it would need the same `EP_CORE.esc()` treatment as `history.html`. Not fixed now to avoid touching a dev-only diagnostic tool's behavior without being asked; flagging for a decision on whether it should be tightened or removed from the production checkout entirely.
- **`ep_upload_avatar`'s image-type detection** (`ep_r2.php`) only content-sniffs for the PNG magic-byte signature (`substr($data,0,4) === "\x89PNG"`) and otherwise assumes JPEG — it does not verify the "otherwise JPEG" branch is actually a valid JPEG (or any valid image at all) before writing it to disk. This is **not currently exploitable as XSS or RCE**, because the stored extension is always hardcoded to `.png` or `.jpg` from a fixed two-value set (never client-influenced), so even non-image bytes saved under a `.jpg` extension would not be served as executable/script content by a normally-configured Apache host. Still, a real magic-byte check for JPEG (`\xFF\xD8\xFF`) would be cheap, consistent insurance matching the rigor already applied to the PNG case — left as an optional hardening item rather than a fix, since it isn't a demonstrated vulnerability today.

---

4. Session/cookie security

Checked: every `setcookie(`/`session_start(`/`session_set_cookie_params(` call under `Etel.Pay/wallet/`; `v2_verify.php`'s `ep_auth()`/`ep_auth_id()`/`ep_get_bearer()` functions; the

`ep_login` / `ep_logout` / `ep_change_pin` token lifecycle; whether any token refresh/expiry-bypass path exists.

Findings

None. No cookie-based session exists anywhere in this codebase, and none was needed to be added. `grep -rn "setcookie(" Etel.Pay/wallet/` returns zero results across the whole tree — confirmed directly rather than assumed.

EtelPay's entire auth model is a real, server-side, database-backed opaque bearer token — not a self-encoding JWT-style token like Delivery & Ride's `gen_token()` / `verify_token()` HMAC scheme. Documented from the literal code:

- `ep_login` (`ep_r1.php`) generates `$token = bin2hex(openssl_random_pseudo_bytes(32))` (a genuinely random, unguessable 256-bit value — not derived from any signable/ forgeable input) and inserts a row into `EP_TBL_SESSIONS` (`ep_sessions`) mapping that token to `ep_user_id` and an `expires_at` timestamp (`time() + EP_SESSION_TTL`, `config.php` sets `EP_SESSION_TTL = 604800` — 7 days).
- `ep_auth()` (`v2_verify.php`) validates a token by joining `ep_sessions` to `ep_users` on the exact token string and requiring `expires_at > NOW()` — since the token itself carries no embedded data and cannot be self-verified offline, revoking access means deleting the DB row, not reasoning about a signature; there is no forgeable-token class of bug here at all (nothing to HMAC-bypass), a structurally different (and in some ways stronger) trust model than Delivery & Ride's signed-token scheme.
- **Real server-side revocation on both logout and PIN change.** `ep_logout` (`ep_r1.php`) deletes the session row for the presented token. `ep_change_pin` (`ep_r2.php`) deletes **every** session row for that user (`DELETE FROM ep_sessions WHERE ep_user_id = ?`) after a successful PIN change — forcing re-login on every device, not just the one that changed the PIN, closing the window for a stolen-but-not-yet-expired token to keep working after the legitimate owner secures their account. `ep_admin_delete_user` (`ep_r2.php`) does the same when an admin closes an account.
- **On expiry: a clean 401, no silent refresh.** No token-refresh action exists anywhere in `ep_r1.php` / `ep_r2.php` / `ep_r3.php` (grepped) — an expired or invalid token always produces `ep_err('Session expired or invalid', 401)` from `ep_auth()`, with no fallback path that re-issues a working token without the client re-authenticating.
- **Account-suspension is checked on every authenticated request, not just at login.** `ep_auth()` re-checks `$row['status'] !== 'active'` (→ 403 "Account suspended") on every call, not only at `ep_login` time — an admin suspending a user via `ep_admin_toggle_user` takes effect on that user's very next API call even if their bearer token is still technically unexpired, without needing a separate session-kill step.
- **`ep_login_rate_limited()` / `ep_record_failed_login()`** (`v2_verify.php`, described in its own comment as fixing a real gap: *"ep_login (a 4-digit PIN, per the audit) had zero attempt tracking or lockout — brute-forceable in minutes"*) enforce a real lockout: 5 failed attempts per phone in 15 minutes, 20 failed attempts per IP in 1 hour, backed by a self-healing `ep_login_attempts` table. Confirmed this is genuinely wired into `ep_login` / `etel_pay_login` (checked before the PIN comparison, and a failed comparison records an attempt via `ep_record_failed_login`) — not just defined and unused.

Verified clean

- No cookie of any kind is set anywhere in this codebase — Secure/HttpOnly/SameSite flags are not applicable because there is no cookie, stated explicitly rather than assuming a mechanism that doesn't exist.

- Session tokens are opaque, server-verified, cryptographically random, and stored/ revoked in a real database table — not a bearer token whose validity can be reasoned about purely client-side.
- Logout and PIN-change both perform genuine server-side revocation (PIN-change additionally revokes every other device's session, which is the more conservative and arguably more correct choice for a wallet app than Delivery & Ride's per-token `token_version` bump — both achieve the same "force re-login everywhere" outcome, via different mechanisms).
- Login brute-force is genuinely rate-limited server-side (confirmed wired in, not just present as dead code), and this pass additionally extended the same limiter to `ep_send_tg_otp` (Section 1/6 — an unauthenticated action that previously had none at all).

Needs Dagi's judgment call

7-day session TTL with no idle/inactivity timeout. `EP_SESSION_TTL = 604800` is a flat 7-day window from issuance regardless of activity — there is no separate "last used" tracking or shorter inactivity-based expiry the way EtelID's own SSO cookie session enforces an 8-hour inactivity cutoff (per the sibling Delivery & Ride audit's Section 4). For a wallet app that moves real money, a shorter idle timeout (in addition to, not instead of, the 7-day hard cap) is a defensible hardening step — but it's a genuine UX/security tradeoff (how often should a legitimate user have to re-enter their PIN) that depends on product judgment, not something to guess at and change unilaterally.

---

5. Error handling and logging sweep

Checked: all eleven PHP files under `Etel.Pay/wallet/api/` for `catch` blocks with a genuinely empty or comment-only body, and for any code path that could leak sensitive internal state (DB credentials, stack traces, secret fragments) into an HTTP response.

Fixed — the `etelid_members_list.php` debug-fingerprint key leak (Section 1's

companion finding, documented here under its actual category: information disclosure via error response, not empty-catch)

Finding (Medium/High severity — narrows a shared secret's effective entropy):
`etelid_members_list.php`'s service-key check, on a failed match, previously returned:

```
eml_err('Invalid or missing service key. debug=' . json_encode($dbg), 401);
```

where `$dbg` included `expected_len`, `expected_prefix` (first 6 chars of the *real* `ETELID_SERVICE_KEY`), and `expected_suffix` (last 6 chars of the real key) — introduced, per its own comment, as a *"temporary debug fingerprint ... safe to remove later"* that was never removed. `ETELID_SERVICE_KEY` is the same shared secret that gates `service_nfc_charge.php` — a **money-moving** endpoint (debits the payer, credits the receiver on an NFC tap) — and `service_check_wallet.php` and `etelid_member_detail.php` (both of which return per-user KYC/status/transaction data). Any unauthenticated caller could send one request with a wrong key to this endpoint and receive back the real key's exact length plus its first and last 6 characters — for a shared secret that this project's own conventions describe as typically 32+ characters, that is roughly a third of the key disclosed outright,

collapsing a brute-force attempt against the remaining unknown middle segment from computationally infeasible to a realistic target, with the exact length removing any need to also guess that.

Fix applied in `Etel.Pay/wallet/api/etelid_members_list.php`: the debug fingerprint was removed entirely; the endpoint now returns only the generic `'Invalid or missing service key.'` message on a mismatch, matching the behavior every other service-key-gated file in this codebase (`etelid_member_detail.php`, `service_check_wallet.php`, `service_nfc_charge.php`) already had. Verified with `php -l` (no syntax errors) and the ASCII-byte scan (no non-ASCII bytes present before or after — this file was already pure ASCII).

Findings — unauthenticated diagnostic endpoints (folded into Section 1's Item,

documented here under its logging/disclosure angle)

`ep_diag_db`, `ep_diag_sessions`, and `ep_diag_telebirr` (`ep_r1.php`) carried a comment claiming they were *"admin-only in production — IP-gated here for dev"*, but no IP gate, admin check, or any other auth existed in the code — confirmed by reading the full case bodies. Before this pass, any unauthenticated caller could retrieve every table's exact row count (including `ep_users` and `ep_transactions` — a rough live user-count and transaction-volume signal for a production wallet), the configured `DB_HOST` / `DB_NAME` (not the password, but real infrastructure fingerprinting information), the count of currently-active vs. expired sessions, and whether Telebirr payment-gateway credentials are configured. Fixed by adding `_ep_require_admin();` as the first line of all three cases (Section 1, Item — same fix, cross-referenced here since it is fundamentally an error-handling/disclosure issue: the comment promised a security control that the code never actually implemented).

Verified clean

- The large majority of `catch` blocks across all eleven files either `ep_err(...)` with a clear message (visible failure, not swallowed) or are documented lazy-schema-migration guards (`etelid_kyc.php`'s `ALTER TABLE ... ADD COLUMN` probe pattern, `ep_r1.php`'s `ep_qr_generate` column-existence probe) — an established, intentional pattern in this codebase, correctly left untouched.
- `updater-api.php`'s `upd_log()` already wraps its own audit-log INSERT in a `try/catch` with an empty body — this mirrors a finding fixed in the sibling Delivery & Ride audit (`log_updater_action()` there had the same silent-audit-trail-failure shape and was given an `error_log()` call). Flagged here as a parallel gap rather than silently matched, but **not changed in this pass** — see "Needs Dagi's judgment call" below, since duplicating that exact fix here is a reasonable follow-up but wasn't in the explicit scope of what broke in this audit.
- No stack traces, raw exception messages containing file paths, or database credentials were found leaking into any HTTP response body in normal operation. `ep_db()`'s own `catch` block calls `ep_err('Database connection failed', 503)` — a generic message, not `$ex->getMessage()` (which could echo the DSN/credentials on a connection failure).

Needs Dagi's judgment call

`updater-api.php`'s `upd_log()` silently swallows a failure to write its own deploy audit trail (`catch (Exception $e) {}`), the same shape as a finding already fixed for Delivery & Ride's equivalent function in the sibling audit (`error_log()` added there). Applying the identical one-line fix here (`error_log('[ETELPAY] upd_log failed: ' . $e->getMessage());`) would be consistent, but is left as a flagged parallel rather than

auto-applied, since it's an incidental finding outside this pass's originally reported scope rather than something independently confirmed broken in production.

---

6. Input validation audit

Checked: every action across all eleven PHP files for how request values are validated before reaching SQL, a filesystem write, or an amount/balance calculation. Particular focus on amount validation (negative/zero/overflow), phone-number handling, and the base64 image-upload paths (`ep_upload_avatar`, `ep_kyc_submit`).

Fixed — `ep_send_tg_otp` had no format validation on the caller-supplied "OTP" and

no rate limiting at all (Medium — open message-relay via the official bot)

Finding: `ep_send_tg_otp` (`ep_r1.php`) is a deliberately unauthenticated action (it runs before the caller has a session — used during password reset and large-transfer confirmation, per its own comment) that accepts a `phone` and an `otp` value **the caller supplies directly**, and relays it, wrapped in a fixed HTML template (`Code: <code>{otp}</code>`), through Etel Group's official Telegram bot to whatever `tg_chat_id` EtelID has on file for that phone. The `otp` parameter was validated only for non-emptiness (`if (!$phone || !$otp) ep_err(...)`), not for actually looking like an OTP code, and the action had **no rate limiting of any kind** — unlike `ep_login`, which reuses the same `ep_login_attempts` table/limiter this action sits right next to in the same file. Together, this meant anyone could use this endpoint to send arbitrary short text, any number of times, to any phone number with Telegram linked, through the project's own official bot — a spam/harassment/phishing relay wearing the bot's real identity, not merely a cosmetic input-validation gap.

Fix applied in `Etel.Pay/wallet/api/ep_r1.php`'s `ep_send_tg_otp`:

```
if (!preg_match('/^\d{4,8}$/', $otp)) ep_err('otp must be a 4-8 digit code');
$ip = isset($_SERVER['REMOTE_ADDR']) ? $_SERVER['REMOTE_ADDR'] : '';
if (ep_login_rate_limited($phone, $ip)) ep_err('Too many attempts. Please wait a while and t
ep_record_failed_login($phone, $ip);
```

`$otp` must now actually be a 4–8 digit numeric code, and the action reuses the exact same per-phone (5/15min)/per-IP (20/hour) limiter `ep_login` already relies on, recording an attempt on every call so the limiter actually trips for repeated use of this action alone (not only when combined with failed logins). Verified with `php -l` (no syntax errors) and the ASCII-byte scan (no new non-ASCII bytes; the pre-existing Amharic/box-drawing characters in this file's comments are unaffected).

Verified clean

- **Amount validation is consistent and real across every money-moving action.** `ep_send` / `ep_escrow_send` require `$amount > 0` and `>= 1` ETB minimum; `ep_platform_transfer` / `ep_partner_credit` require `> 0` plus an explicit per-call cap (`EP_PARTNER_CREDIT_MAX` = 3000 ETB, `EP_PLATFORM_TRANSFER_MAX` = 50000 ETB) and rolling 24-hour cap per partner; `service_nfc_charge.php` requires `$amount > 0` independently. No action was found that skips a positive-amount check before touching a balance.

- **Phone-number normalization** (`ep_normalize_phone()`, `v2_verify.php`) is applied consistently before every phone-based lookup (`ep_send`'s receiver, `ep_partner_credit`, `ep_platform_transfer`'s `from_phone`/`to_phone`) — the same class of bug flagged and fixed for Delivery & Ride ("a locally-formatted '09...' number silently fails to match the '+251...' row already in the DB") does not recur here because normalization is applied at every phone-lookup call site, confirmed by grep.
- **`ep_upload_avatar`** enforces a 500KB decoded-size cap before writing to disk, and (as covered in Section 3) the stored extension is always one of two hardcoded values, never client-supplied.
- **`ep_kyc_submit`** enforces a 3MB cap on the front-document base64 payload before insertion (`strlen($doc_b64) > 4194304`, accounting for base64 overhead) — see "Needs Dagi's judgment call" below for a minor asymmetry noticed here.
- **The Updater's file-deploy path** (`updater-api.php`) whitelists both destination extensions (`$ALLOWED_EXT` for text-based deploys, plus a separate image/zip whitelist for `upload_file`) and uses `realpath()`-based traversal checks (`strpos($dest, realpath($BASE_DIR)) !== 0`) consistently across `list_files`, `read_file`, `update_file`, and the ZIP-deploy path — matching the pattern already verified for Delivery & Ride's equivalent tool in the sibling audit. `config.php` and `.env` are additionally hardcoded into `$PROTECTED_FILES` and can never be overwritten by any Updater action, regardless of the uploaded filename.
- **`upd_read_zip_entries()`'s pure-PHP ZIP parser** — per the shared Etel Group ops knowledge, this parser previously had a genuine End-Of-Central-Directory byte-offset bug; confirmed the currently-checked-out version reads `total` and `offset` as two separate unpacked fields (`unpack('vtotal/Voffset', ...)`) at the correct relative offsets, consistent with that bug already being fixed in this codebase, not a fresh regression.

Needs Dagi's judgment call

- **`ep_kyc_submit`'s size cap is only checked on `doc_front_b64`**, not on `doc_back_b64` or `selfie_b64` — all three are stored as `LONGTEXT` (unbounded at the application layer, only bounded by MySQL's own `LONGTEXT` ceiling), so a caller could submit a very large `doc_back_b64`/`selfie_b64` without the same 3MB-equivalent check the front document gets. This is not itself exploitable beyond an oversized-request nuisance (no path traversal, no injection), and mirrors a similarly minor, non-blocking gap flagged (not fixed) in the sibling Delivery & Ride audit's Section 6 (`push_subscriptions` columns with no explicit max-length check). Left as an optional consistency fix rather than a genuine vulnerability.
- **`register.html`'s registration form posts to a non-existent action** (`etel_pay_register_wallet`) — flagged under Section 1 as the reason this codebase has no client-controlled-role bug, but restated here as a straightforward functional defect: self-service registration is currently broken end-to-end on the standalone EtelPay app (only the EtelID-SSO auto-provisioning path in `ep_login_etelid` actually creates new wallets). Whether to wire up a real `register` action, deliberately keep registration EtelID-SSO-only as a design choice (consistent with the "EtelID is the identity root" principle this whole group's ops model is built around), or remove the dead form entirely is a product decision, not something to guess at and build unasked.

---

7. Known open items — confirmed current status

7.1 `backups/` directory web-accessibility

Checked: `Etel.Pay/wallet/backups/` — unlike Delivery & Ride's equivalent (created lazily at runtime, empty in this repo checkout), EtelPay's `backups/` directory is **checked into this repo with 222 real timestamped backup files already in it** (`updater-api.php`'s `upd_backup()` writes here before every file overwrite via the Updater — e.g. `20260617_220922_js_config.js`, `20260614_212955_wallet_wallet.html`).

Found: the same gap already identified and fixed for Delivery & Ride's `api/backups/` in the sibling audit exists here too: `Etel.Pay/wallet/api/.htaccess` (covering `api/`) only protects `config.php` and `.env` by filename via `<Files>` directives — it does not cover `Etel.Pay/wallet/backups/` at all (a sibling directory to `api/`, not inside it, so `api/.htaccess`'s rules don't apply there regardless). No `.htaccess` existed inside `Etel.Pay/wallet/backups/` itself before this pass. A direct request to e.g. `https://ethelmarket.com/Etel.Pay/wallet/backups/20260614_212955_wallet_wallet.html` was not explicitly denied by anything.

Why this matters more here than in Delivery's case: because EtelPay's `backups/` directory already has 222 real files checked in (not an empty runtime-created folder), this was an immediately live, currently-exploitable gap, not a theoretical one — any of those 222 timestamped files (past versions of `wallet.html`, `admin.html`, `js/config.js`, etc.) was directly fetchable by URL. `api/config.php` and `api/.env` are both hardcoded into `updater-api.php`'s `$PROTECTED_FILES` list and can never be overwritten, so they have never been backed up here (confirmed: `grep -i config` against the backup filenames turns up only `js/config.js`, never `api/config.php`) — so no database credential or secret was directly exposed through this specific gap. But it did expose historical source of the admin panel, wallet UI, and JS config across many past deploys to anyone who requested a filename (filenames are guessable/enumerable by date pattern, and the directory itself has no `Options -Indexes` protection either before this fix).

Fixed: created `Etel.Pay/wallet/backups/.htaccess`:

```
<IfModule mod_authz_core.c>
    Require all denied
</IfModule>
<IfModule !mod_authz_core.c>
    Order allow,deny
    Deny from all
</IfModule>
Options -Indexes
```

Matches the Apache 2.4+ primary rule with an older-Apache fallback, the same pattern already established at `delivery/api/backups/.htaccess` and `EtelAi/backups/.htaccess` elsewhere in this group's codebase.

7.2 Hardcoded/placeholder secrets in the Updater and money-moving endpoints

Checked: `config.php`'s three placeholder-shipped secrets (`EP_DB_PASS`, `EP_SECRET`, `EP_UPDATER_PASS`) and every place each is actually consumed.

Found — two real placeholder-acceptance bugs, both fixed in this pass (already detailed above, cross-referenced here as this section's namesake item): - `EP_SECRET` — Section 1, Item 1 (already fixed ahead of this session; re-verified here). - `EP_UPDATER_PASS` — Section 1, Item 2 (fixed in this pass). - `EP_DB_PASS` has no such gap: `v2_verify.php` (lines 117–135) already fails closed with a clear 500 error (`'Database not configured. Set DB_PASS in api/.env'`) via `ep_is_placeholder()` if `EP_DB_PASS` is empty or still the

shipped `CHANGE_ME` literal — confirmed correct, no change needed. - `ETELID_SERVICE_KEY` — already correctly guarded (case-insensitive placeholder check) in every file that consumes it (`etelid_member_detail.php` , `etelid_members_list.php` , `service_check_wallet.php` , `service_nfc_charge.php`) — no change needed there beyond the separate debug-leak fix in Section 5.

No changes needed beyond what's already covered above — this section exists to confirm, item by item, that every placeholder-shipped secret in `config.php` now has a real fail-closed or placeholder-rejecting guard at every consumption site, not just some.

---

8. Full API endpoint inventory

All actions are dispatched by `v2_verify.php` 's single entrypoint: the action name is read from `$body['action']` (POST JSON body), falling back to `$body['ep_action']` , then `$_GET['action']` . Dispatch tries `ep_route_part1($action)` , then `ep_route_part2($action)` , then `ep_route_part3($action)` in order, each returning `false` to fall through if the action isn't its own — a 404 `'Unknown action'` results only if none of the three claim it. Four additional standalone scripts (Section 8.2) are called directly by URL, bypassing this router entirely. Access level is determined by the actual guard call inside each case: `ep_auth()` / `ep_auth_id()` (any valid session), `_ep_require_admin()` (session + `is_admin` / `admin_phones`), a shared-secret `hash_equals()` check (service-to-service), or no guard (public/unauthenticated by design).

8.1 `ep_r1.php` — `ep_route_part1()`

Action	Access Level
`ep_ping`	Public
`ep_send_tg_otp`	Public (rate-limited + OTP-format-checked — this pass)
`ep_check_tg_link`	Public
`ep_diag_db`	Admin (`_ep_require_admin()` — this pass)
`ep_diag_sessions`	Admin (this pass)
`ep_diag_telebirr`	Admin (this pass)
`ep_login` , `etel_pay_login`	Public (rate-limited)
`ep_logout` , `etel_pay_logout`	Public (no-ops cleanly with no/invalid token)
`ep_get_balance` , `etel_pay_get_balance`	Authenticated
`ep_get_history` , `etel_pay_get_history`	Authenticated (scoped to caller's own tx)
`ep_get_rates` , `etel_pay_get_rates`	Public
`ep_send` , `etel_pay_send`	Authenticated
`ep_escrow_send`	Authenticated
`ep_escrow_release`	Service key (`X-Etel-Key` = `EP_SECRET_FINAL`)

Action	Access Level
`ep_escrow_reverse`	Service key
`ep_verify_tx`, `etel_pay_verify_tx`	Service key
`ep_partner_credit`	Service key
`ep_platform_transfer`	Service key
`ep_qr_generate`, `etel_pay_qr_generate`	Authenticated
`ep_qr_process`, `etel_pay_qr_process`	Authenticated

8.2 Standalone service-to-service scripts (bypass `v2_verify.php` entirely)

File	Access Level
`etelid_member_detail.php`	Service key (`X-Etel-Service-Key` header or `?key=`)
`etelid_members_list.php`	Service key
`service_check_wallet.php`	Service key
`service_nfc_charge.php`	Service key

8.3 `ep_r2.php` — `ep_route_part2()`

Action	Access Level
`ep_get_profile`, `ep_portal_get_profile`	Authenticated
`ep_update_profile`	Authenticated
`ep_change_pin`	Authenticated (revokes all sessions on success)
`ep_upload_avatar`	Authenticated
`ep_admin_stats`	Admin
`ep_admin_users`	Admin
`ep_admin_get_user`	Admin
`ep_admin_toggle_user`	Admin
`ep_admin_credit`	Admin
`ep_admin_self_topup`	Admin *(see Section 1 judgment call)*
`ep_admin_debit`	Admin
`ep_admin_txs`	Admin
`ep_admin_reverse_tx`	Admin (idempotent status-flip guard)
`ep_admin_search`	Admin
`ep_admin_rates`	Admin

Action	Access Level
`ep_admin_update_rate`	Admin
`ep_admin_settings`	Admin
`ep_admin_save_settings`	Admin
`ep_admin_delete_user`	Admin (revokes all sessions)
`ep_topup_initiate`	Authenticated
`ep_topup_confirm`	Authenticated (scoped to caller's own top-up request)
`ep_admin_topup_approve`	Admin (idempotent status-flip guard)
`ep_admin_topup_list`	Admin

8.4 `ep_r3.php` — `ep_route_part3()`

Action	Access Level
`ep_paylink_create`	Authenticated
`ep_paylink_info`	Public (by design — a payment link's whole purpose is to be
opened by an unauthenticated payer before they log in)	
`ep_paylink_pay`	Authenticated
`ep_paylink_list`	Authenticated (scoped to caller's own links)
`ep_apikey_create`	Authenticated
`ep_apikey_list`	Authenticated (scoped)
`ep_apikey_revoke`	Authenticated (scoped — `WHERE id=? AND user_id=?`)
`ep_kyc_submit`	Authenticated
`ep_admin_kyc_list`	Admin
`ep_admin_kyc_review`	Admin
`ep_analytics`	Admin
`ep_my_analytics`	Authenticated (scoped to caller)
`ep_dispute_open`	Authenticated
`ep_admin_dispute_list`	Admin
`ep_admin_dispute_resolve`	Admin (idempotent status-flip guard added — this pass)
`ep_etelid_kyc_sync`, `etel_pay_etelid_kyc_sync`	Authenticated
`ep_etelid_resolve`	Authenticated

Action	Access Level
`ep_login_etelid`	Public (EtelID SSO exchange — auto-provisions a wallet)
`ep_nonce`, `etel_pay_nonce`	Public

8.5 `updater-api.php` — separate router, own auth scheme (password + signed token)

Action	Access Level
`ping`	Public
`login`	Public (password check, rejects placeholder — this pass)
`list_files`	Updater token
`read_file`	Updater token
`update_file`	Updater token (blocks `config.php` / `.env`)
`upload_file`	Updater token (blocks `config.php` / `.env`; delegates to ZIP path for `.zip`)
`upload_zip`	Updater token
`upload_multiple`	Updater token
`list_backups`	Updater token
`restore_backup`	Updater token
`delete_backup`	Updater token
`run_sql`	Updater token *(see Section 1 judgment call — arbitrary SQL)*
`info`	Updater token

Grand total: ~66 distinct action names across `ep_r1.php` (19) + `ep_r2.php` (23) + `ep_r3.php` (19) + the 4 standalone service scripts + `updater-api.php` (13, separate auth scheme).

---

9. Sample request/response pairs

All based directly on the actual route-handler code (field names/types exactly as read via `in_str()` / `in_float()` / `in_int()` and returned via `ep_ok($data, $message)`, which wraps every successful response as `{"success": true, "data": {...}, "message": "...", "v": "<EP_VER>"}`, and `ep_err($message, $code)`, which returns `{"success": false, "data": {"message": "..."}, "v": "<EP_VER>"}` with the given HTTP status code). Placeholder values below are fictitious — no real customer data.

9.1 Login — `ep_login`

(`ep_r1.php`, no auth required, rate-limited)

Request (POST `api/v2_verify.php`):

```
{
  "action": "ep_login",
  "phone": "0911000000",
  "pin": "1234"
}
```

Success response:

```
{
  "success": true,
  "data": {
    "token": "a1b2c3d4e5f6...<64-hex-char opaque bearer token>",
    "expires_at": "2026-09-27 12:00:00"
  },
  "message": "Login successful",
  "v": "1.1.0"
}
```

(`token` is used as `Authorization: Bearer <token>` on every subsequent authenticated call. Rate-limited: 5 failed attempts per phone in 15 minutes, 20 failed attempts per IP in 1 hour, both return HTTP 429.)

9.2 Send money — `ep_send`

(`ep_r1.php`, requires `Authorization: Bearer <token>`)

Request:

```
{
  "action": "ep_send",
  "receiver": "0922000000",
  "amount": 150.00,
  "currency": "ETB",
  "note": "Lunch"
}
```

Success response:

```
{
  "success": true,
  "data": {
    "tx_ref": "EP-A1B2C3D4E5F6",
    "amount": 150.00,
    "currency": "ETB",
    "receiver_phone": "+2519220000000",
    "receiver_name": "Test User",
    "note": "Lunch"
  },
  "message": "Transfer successful",
```

```
  "v": "1.1.0"
}
```

(Fails with a 400 "Insufficient balance" if the sender's `balance_etb` is below the requested amount at read time, or a 500 "Transfer failed: Insufficient balance (concurrent change)" if a concurrent request already spent the balance between the pre-check and the guarded `UPDATE`.)

9.3 Partner credit (server-to-server) — `ep_partner_credit`

(`ep_r1.php`, requires `X-Etel-Key: <EP_SECRET_FINAL>` header — not a user session)

Request:

```
{
  "action": "ep_partner_credit",
  "phone": "0933000000",
  "amount": 250.00,
  "partner": "delivery",
  "partner_ref": "DEL-DRIVER-PAYOUT-2026-09-20-001",
  "note": "Driver earnings payout"
}
```

Success response:

```
{
  "success": true,
  "data": {
    "tx_ref": "EP-B2C3D4E5F6A1",
    "amount": 250.00,
    "phone": "+251933000000"
  },
  "message": "Credited",
  "v": "1.1.0"
}
```

(A retried call with the same `partner` + `partner_ref` returns `{"idempotent_replay": true, ...}` instead of crediting twice. Fails with 400 if `amount` exceeds the 3000 ETB per-call cap, or 429 if the partner's rolling 24-hour total would exceed 20000 ETB.)

9.4 NFC tap-to-pay charge (server-to-server) — `service_nfc_charge.php`

(standalone script, not through `v2_verify.php`; requires `X-Etel-Service-Key: <ETELID_SERVICE_KEY>`; called by EtelID only after both the receiver and the payer have independently verified their own EtelID password on-device)

Request (POST `api/service_nfc_charge.php`):

```
{
  "receiver_phone": "+251911000000",
  "payer_phone": "+251922000000",
  "amount": 45.00,
  "note": "Coffee"
}
```

Success response:

```
{
  "success": true,
  "data": {
    "tx_ref": "NFC1A2B3C4D5E6F",
    "amount": 45.00,
    "currency": "ETB"
  }
}
```

(Fails with 402 "Insufficient balance." if the payer's wallet balance is below the requested amount, 404 if either phone has no EtelPay wallet, or 403 if either wallet is suspended. This endpoint's response shape is flat `{success, data}` — no `v` field — since it does not go through `v2_verify.php`'s `ep_ok()` helper.)

Summary of code changes made in this pass

File	Change
`Etel.Pay/wallet/api/ep_r1.php`	Added `_ep_require_admin()` gate to `ep_diag_db` / `ep_diag_sessions` / `ep_diag_telebirr` (were fully unauthenticated); added OTP-format validation + rate limiting to `ep_send_tg_otp` (was an unauthenticated, unlimited Telegram-relay). *(`EP_SECRET` → `EP_SECRET_FINAL` on the 5 money-moving service actions was already fixed ahead of this pass — re-verified only, not re-edited.)*
`Etel.Pay/wallet/api/ep_r3.php`	Added an idempotent status-flip guard to `ep_admin_dispute_resolve`'s `refund` branch (previously could double-credit on a retried/double-clicked request)
`Etel.Pay/wallet/api/updater-api.php`	Fixed `upd_password_is_configured()` to reject the shipped placeholder `EP_UPDATER_PASS` value (`CHANGE_ME`) case-insensitively — the prior case-sensitive check against lowercase `changeme` let the literal shipped default through as a "configured" password, granting full file-deploy (arbitrary PHP write) access

File	Change
`Etel.Pay/wallet/api/etelid_members_list.php`	Removed a debug-fingerprint response on service-key mismatch that leaked the real `ETELID_SERVICE_KEY` 's exact length plus its first/last 6 characters
`Etel.Pay/wallet/backups/.htaccess`	**New** — deny-all + `Options -Indexes` lockdown (this directory already had 222 real timestamped backup files checked in with no access protection at all)

All PHP edits verified with `php -l` (no syntax errors) and the ASCII-safety check (no new non-ASCII bytes introduced by any edit — pre-existing Amharic text and box-drawing characters throughout this codebase's comments were left untouched and are not a violation of the ASCII constraint being newly introduced). Nothing was deployed — all changes are in this repo's git working tree only; deployment to `ethelmarket.com/Etel.Pay/wallet/` is a separate manual step through EtelPay's own Updater tool (`updater.html`).

Consolidated list of items needing Dagi's judgment call

1. **`ep_admin_self_topup` lets any admin mint unlimited balance into their own wallet** (Section 1) — a legitimate, by-design admin power (mirrors `ep_admin_credit` for other users), not a bug. Flagged for whether a secondary approval or audit alert is wanted given its blast radius. Not changed.
2. **`run_sql` in `updater-api.php` grants arbitrary SQL execution behind a single shared password** (Section 1) — consistent with this class of tool across Etel Group's other systems, now properly gated by the placeholder-password fix in this pass, but still a high-blast-radius capability with no second factor. Not changed.
3. **`diag.html`'s unescaped `innerHTML` from `JSON.stringify()`'d API responses** (Section 3) — low-risk today (self-triggered, developer-only, not linked from normal navigation), but would need `EP_CORE.esc()` treatment if this page is ever exposed more broadly or starts echoing another party's input. Not changed.
4. **`ep_upload_avatar` only magic-byte-checks for PNG, assumes JPEG otherwise** (Section 3) — not currently exploitable (extension is always hardcoded, never client-supplied), but a real JPEG magic-byte check would be cheap, consistent insurance. Not changed.
5. **7-day session TTL with no separate inactivity/idle timeout** (Section 4) — a real UX/security tradeoff for a wallet app; EtelID's own SSO session enforces an 8-hour idle cutoff by comparison. Not changed — depends on product judgment about how often a legitimate user should have to re-enter their PIN.
6. **`updater-api.php`'s `upd_log()` silently swallows a failure to write its own deploy audit trail** (Section 5) — the same shape as a finding already fixed for Delivery & Ride's equivalent function (`error_log()` added there); applying the identical fix here would be consistent, but was outside this pass's originally reported scope. Not changed.
7. **`ep_kyc_submit`'s size cap is only enforced on the front-document field, not the back-document or selfie fields** (Section 6) — a minor, non-exploitable consistency gap (no injection/traversal risk, `LONGTEXT` is the only real ceiling). Not changed.
8. **`register.html` posts to a non-existent `etel_pay_register_wallet` action** (Sections 1 and 6) — self-service registration is currently non-functional on the standalone EtelPay app; only EtelID-SSO auto-provisioning (`ep_login_etelid`) actually creates wallets today. Whether to build a real registration action,

deliberately keep registration EtelID-SSO-only, or remove the dead form is a product decision. **Resolved — removed (see second pass below).**

Second pass — Dagi's decisions on the judgment-call list

Dagi reviewed the 8 items above and decided:

#	Item	Decision
1	`ep_admin_self_topup` unlimited self-mint	Leave as-is — by-design admin power, no secondary approval added
2	`run_sql` single shared password	Leave as-is — accepted, consistent with the rest of Etel Group
3	`diag.html` unescaped `innerHTML`	**Fixed** — see below
4	`ep_upload_avatar` JPEG not magic-byte-checked	**Fixed** — see below
5	No session idle timeout	**Fixed** — 8-hour idle timeout added
6	`upd_log()` swallows write failures	**Fixed** — see below
7	`ep_kyc_submit` size cap not on back/selfie	**Fixed** — see below
8	Dead `register.html` self-registration form	**Fixed** — form removed

Summary of code changes made in this second pass

File	Change
`Etel.Pay/wallet/diag.html`	Added a local `esc()` HTML-escaping helper and applied it to the raw JSON API-response dump in `log()` (item 3). The developer-authored status markup passed alongside it was left unescaped, since it is trusted local HTML, not response data.
`Etel.Pay/wallet/api/ep_r2.php`	`ep_upload_avatar` now checks the real JPEG magic bytes (`FF D8 FF`) instead of assuming any non-PNG upload is a valid JPEG (item 4).
`Etel.Pay/wallet/api/v2_verify.php`	Added an 8-hour (`28800` s) idle-session timeout: a new `last_active_at` column on `ep_sessions` (added lazily, same self-healing pattern as `etelid_kyc.php`), checked and refreshed on every `ep_auth()` call; a session idle past the cutoff is deleted and rejected with 401 (item 5).
`Etel.Pay/wallet/api/updater-api.php`	`upd_log()` 's empty catch block now calls `error_log()` on a failed write, matching the equivalent fix already made for Delivery & Ride (item 6).
`Etel.Pay/wallet/api/ep_r3.php`	`ep_kyc_submit` 's existing ~3 MB (4194304-byte) size cap is now enforced on the back-document and selfie fields as well as the front document (item 7).

File	Change
`Etel.Pay/wallet/register.html`	**Deleted.** Confirmed orphaned — nothing links to it, and the action it posts to (`etel_pay_register_wallet`) has never existed on the backend. Self-service registration remains EtelID-SSO-only via `ep_login_etelid` (item 8).

All PHP edits verified with `php -l` and the ASCII-safety check (no new non-ASCII bytes introduced). The JS in `diag.html` was verified with `node --check`. Nothing was deployed — all changes are in this repo's git working tree only; deployment to `ethelmarket.com/Etel.Pay/wallet/` remains a separate manual step through EtelPay's own Updater tool (`updater.html`).